

420-301-VA: Web Services

Lab 5: Implementing a RESTful API with PHP and PDO

Computer Science & Technology
Vanier College

Introduction

In Lab 4, you designed a RESTful API and documented its contract using the OpenAPI specification. Now, it is time to build the actual backend service that fulfills that contract.

In this lab, we will build a lightweight REST API from scratch using **PHP** and **MariaDB**. Instead of relying on a heavy framework like Laravel or Symfony, you will manually handle HTTP routing, parse raw JSON requests, and interact with the database using **PDO (PHP Data Objects)**. Building this “from scratch” will give you a profound understanding of how modern web frameworks operate under the hood.

Objectives

By the end of this lab, you should be able to:

- Implement a “Front Controller” routing script to intercept and direct HTTP requests based on the URI and HTTP Method.
- Set correct HTTP headers to return `application/json` responses and handle CORS.
- Read and decode raw JSON payload bodies from HTTP `POST`, `PUT`, and `PATCH` requests.
- Establish a secure connection to a MariaDB database using PDO.
- Execute CRUD operations using **PDO Prepared Statements** to prevent SQL Injection.
- Extract path variables (like IDs) from URIs using Regular Expressions to handle targeted `PUT`, `PATCH`, and `DELETE` requests.

Instructions

- You will need a local web server environment running PHP and MariaDB (e.g., XAMPP, MAMP, or a Docker stack).
- Use a graphical API client like **Postman** or **Insomnia** to test your endpoints as you build them. (A web browser can only easily test **GET** requests).
- Write clean, indented code. Ensure no lines are excessively long to maintain readability.
- Copy-paste your PHP code for each exercise into a separate document, along with screenshots of your Postman requests and responses to prove the endpoints work.

1 Part 1: The Front Controller and JSON Routing

1.1 Concept Review: The Front Controller Pattern

In a traditional, older PHP application, visiting `/books.php` runs one script, and `/users.php` runs another. In modern REST APIs, we use a single entry point (usually `index.php` or `api.php`) called a **Front Controller**. All traffic is directed here. The script inspects the `$_SERVER['REQUEST_URI']` and `$_SERVER['REQUEST_METHOD']` to decide which logic to execute.

Furthermore, REST APIs communicate via JSON.

- **To send JSON:** We must set the `Content-Type` header and convert PHP arrays to JSON strings using `json_encode()`.
- **To receive JSON:** We *cannot* use the standard `$_POST` array (which only works for HTML forms). Instead, we must read the raw HTTP request body using `file_get_contents("php://input")`.

Exercise 1: Building the Router Skeleton

Step 1A: Setup the Database (MariaDB)

- Open phpMyAdmin (or your preferred database client).
- Create a database named `library_db`.
- Run the following SQL to create a `books` table and insert initial mock data:

```
CREATE TABLE books (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    author VARCHAR(255) NOT NULL,  
    is_available BOOLEAN DEFAULT TRUE  
);  
  
INSERT INTO books (title, author) VALUES ('1984', 'George Orwell');  
INSERT INTO books (title, author) VALUES ('Dune', 'Frank Herbert');
```

Step 1B: The Routing Logic Create a file named `api.php` in your web server's document root (e.g., `htdocs` or `www`). Add the following skeleton code. Read the comments carefully to understand how the request is intercepted.

```
<?php  
// 1. Set headers to allow JSON and handle CORS  
header("Access-Control-Allow-Origin: *");  
header("Content-Type: application/json; charset=UTF-8");  
header("Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE");  
  
// 2. Identify the HTTP Method and URI  
$method = $_SERVER['REQUEST_METHOD'];  
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);  
  
// 3. Basic Routing Logic  
if ($uri === '/api.php/books') {
```

```

if ($method === 'GET') {
    // Handle GET Request
    echo json_encode(["message" => "GET all books endpoint hit"]);
} elseif ($method === 'POST') {
    // Handle POST Request: Read the raw JSON body
    $rawJson = file_get_contents('php://input');

    // Decode JSON into a PHP associative array
    $input = json_decode($rawJson, true);

    echo json_encode([
        "message" => "POST endpoint hit",
        "received_data" => $input
    ]);
}
} else {
    // 4. Handle 404 Not Found for unknown URIs
    http_response_code(404);
    echo json_encode(["error" => "Endpoint not found"]);
}
?>

```

Step 1C: Testing the Router in Postman

- Open Postman.
- Make a GET request to `http://localhost/api.php/books`. Verify you receive the JSON message.
- Make a POST request to the same URL.
 - In Postman, go to the **Body** tab.
 - Select **raw** and ensure the dropdown is set to **JSON**.
 - Send the following payload: `{"title": "Test Book"}`.

Verify the API echoes your data back in the response.

- Make a GET request to `http://localhost/api.php/unknown`. Verify you get a 404 Not Found status code and the error message.

2 Part 2: Integrating PDO for GET and POST

2.1 Concept Review: PHP Data Objects (PDO)

PDO is a robust, object-oriented database abstraction layer in PHP. Its most important feature is **Prepared Statements**. When handling user data (like a POST request), you must never insert variables directly into an SQL string. Prepared statements separate the SQL logic from the user input, completely preventing SQL Injection attacks.

Exercise 2: Implementing Database Operations

Step 2A: The Database Connection At the top of your `api.php` file (right under the headers), instantiate a PDO connection.

```
// Database Configuration
$host      = '127.0.0.1';
$db        = 'library_db';
$user      = 'root'; // Update with your DB credentials if different
$pass      = '';     // Update with your DB credentials if different
$charset   = 'utf8mb4';

// Data Source Name
$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

// PDO Options
$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES  => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
    http_response_code(500); // 500 Internal Server Error
    echo json_encode(["error" => "Database connection failed"]);
    exit;
}
```

Step 2B: Implementing GET (Read All) Go back to your routing block. Replace the GET placeholder inside your routing if statement with actual database retrieval logic.

```
if ($method === 'GET') {
    // Query the database
    $stmt = $pdo->query('SELECT * FROM books');
    $books = $stmt->fetchAll();

    // Return results as JSON
    http_response_code(200);
    echo json_encode($books);
}
```

Test this in Postman. You should now see the two books you inserted via SQL in Step 1A returned as a JSON array!

Step 2C: Implementing POST (Create) Replace the POST placeholder. You must extract the title and author from the JSON array, validate them, prepare an INSERT statement, and execute it safely.

```
} elseif ($method === 'POST') {  
    // Read and decode the input  
    $rawJson = file_get_contents('php://input');  
    $input = json_decode($rawJson, true);  
  
    // Basic Validation: Ensure fields exist  
    if (!isset($input['title']) || !isset($input['author'])) {  
        http_response_code(400); // 400 Bad Request  
        echo json_encode(["error" => "Missing title or author"]);  
        exit;  
    }  
  
    // Use Prepared Statements to prevent SQL injection!  
    $sql = "INSERT INTO books (title, author) VALUES (:title, :author)";  
    $stmt = $pdo->prepare($sql);  
  
    // Execute by binding the parameters safely  
    $stmt->execute([  
        'title' => $input['title'],  
        'author' => $input['author']  
    ]);  
  
    // Respond with success and the new ID  
    http_response_code(201); // 201 Created  
    echo json_encode([  
        "message" => "Book created successfully",  
        "id" => $pdo->lastInsertId()  
    ]);  
}
```

Test this in Postman by sending a POST request with a new book (e.g., "Foundation" by "Isaac Asimov"). Then run your GET request again to verify it was saved to the database permanently.

3 Part 3: Advanced Routing (PUT, PATCH, DELETE)

3.1 Concept Review: Parsing Path Variables

To update or delete a specific resource, the client sends the ID directly in the URL (e.g., DELETE /api.php/books/42). Our simple if (\$uri === '/api.php/books') check will fail here because the URI is different.

We need to extract the ID from the URI. The cleanest way to do this without a framework is by using a **Regular Expression** (preg_match).

- **PUT** implies replacing the entire resource (updating all fields).
- **PATCH** implies a partial update (e.g., only flipping is_available to false).
- **DELETE** removes the resource.

Exercise 3: Targeted Operations

Step 3A: Regex Routing for IDs Add an elseif block to your main routing logic to catch URIs that contain an ID at the end.

```
// ... (Right below the closing bracket of your first routing block)

// Check if URI matches the pattern /api.php/books/{id}
// The ([0-9]+) captures the numerical ID into the $matches array.
} elseif (preg_match('/^\/api\.php\/books\/([0-9]+)$/ ', $uri, $matches)) {

    // The ID is captured in $matches[1]
    $bookId = (int)$matches[1];

    if ($method === 'GET') {
        // TODO: Implement fetch single book by ID
    } elseif ($method === 'DELETE') {
        // TODO: Implement delete book
    } elseif ($method === 'PATCH') {
        // TODO: Implement partial update
    }
}
```

Step 3B: Your Turn - Implement DELETE Inside the DELETE block above, write the PDO logic to delete the book with the given `$bookId`. Use the following pedagogical steps to guide you:

1. Prepare a SQL statement: `DELETE FROM books WHERE id = :id`.
2. Execute it, binding the `$bookId` variable to the `:id` parameter.
3. Check `$stmt->rowCount()`. If it returns 0, it means no rows were deleted (the book didn't exist). Return a `404 Not Found` response.
4. If successful (`rowCount > 0`), return a `200 OK` with a JSON success message.
5. **Test it:** Use Postman to send a DELETE request to `http://localhost/api.php/books/1`. Attempt to delete it a second time to ensure you receive your `404` message!

Step 3C: Your Turn - Implement PATCH (Partial Update) Many library systems need to “check out” a book. We don't want to replace the whole book record (so we won't use PUT), we just want to change the `is_available` boolean.

Inside the PATCH block:

1. Read the JSON input using `file_get_contents` and `json_decode`.
2. Check if the `is_available` key exists in the input array.
3. Prepare an `UPDATE books SET is_available = :status WHERE id = :id` statement.
4. Execute it, binding both the `$bookId` and the new status.
5. Return a success message.
6. **Test it:** Use Postman to send a PATCH request to `http://localhost/api.php/books/2` sending the payload `{"is_available": 0}`. Run a GET request to verify the database was updated!

References and Further Reading

- [PHP Official Documentation: PDO](#): The definitive guide to connecting to databases, preparing statements, and fetching data securely in PHP.
- [PHP: json_encode and json_decode](#): Documentation on converting between PHP associative arrays/objects and JSON strings.
- [MDN Web Docs: HTTP request methods](#): A detailed breakdown of the semantics of GET, POST, PUT, PATCH, and DELETE methods.
- [PHP The Right Way - Databases](#): An excellent community-driven guide on best practices for database interaction and why Prepared Statements are mandatory for modern web security.